



Python应用基础

第13章 Matplotlib数据可视化

陈刚

中山大学岭南学院

数据分析篇目录

Contents

第 11 章

NumPy 基础

第 12 章

Pandas 金融数据分析

第 13 章

Matplotlib 数据可视化

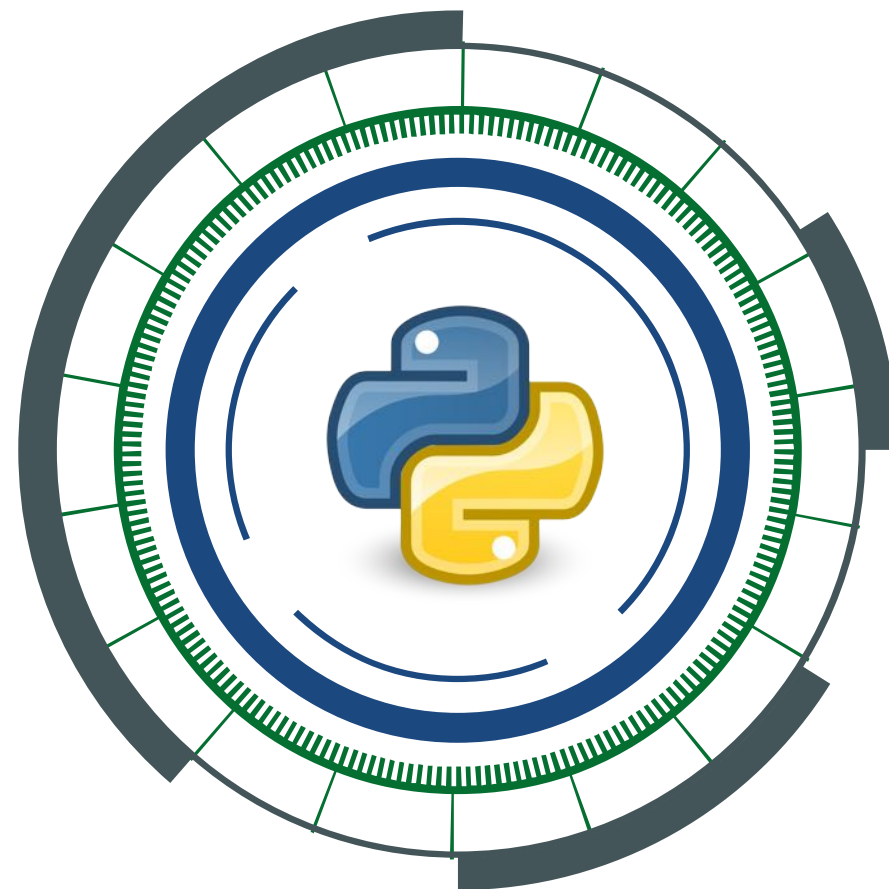


pythonTM
programming



学习目标

- 1 熟悉数据可视化中图表的各类元素
- 2 学会利用 Matplotlib 绘制常见图形进行数据描述





13.1

概述

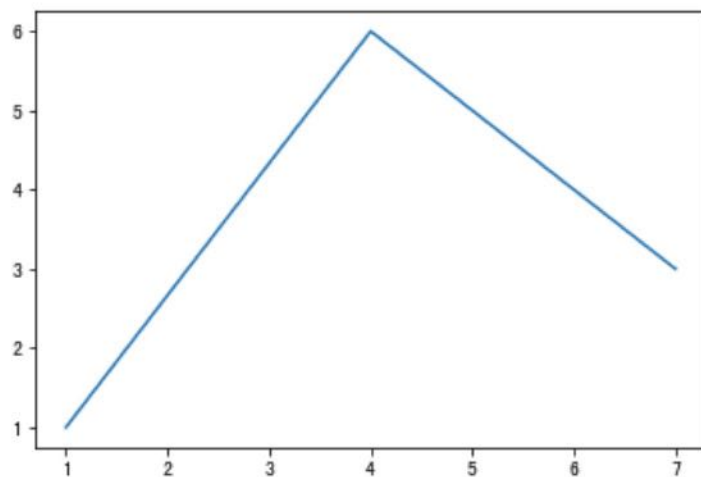
概述

- Matplotlib 是一个 2D 图形库，能生成各式图形。其界面具有可交互性，同时 Matplotlib 库又能够跨平台，既可在 Python 脚本中进行编码操作，也能在 Jupyter Notebook 中使用，在其他相关的编辑平台也可以便捷使用。
- 在 Jupyter Notebook 平台使用，相关图形库已在软件内直接配置，但是其生成的图形无法进行交互，而是单独内嵌于 Jupyter Notebook 的界面中。
- 在金融应用领域内，掌握 Matplotlib 技能是必备的，Matplotlib 能将数据进行可视化，更直观地呈现，使数据更加美观，更有说服力。

案例

【例】 以一个简单的示例来感受下 Matplotlib 的魅力。

```
① import matplotlib.pyplot as plt  
② plt.figure()  
③ plt.plot([1, 4, 6, 7], [1, 6, 4, 3])  
④ plt.show()
```



1. plt.figure() — 创建画布

- plt.figure() 用于创建一个新的图形画布（Figure对象）。
- 如果不调用它，plt.plot() 会自动创建一个默认画布。

基本语法：

```
plt.figure(num=None, figsize=None, dpi=None, facecolor=None, edgecolor=None, ...)
```

参数	说明	示例
num	图形编号或名称（窗口标题）	num=1 或 num="我的图表"
figsize	画布大小，单位为英寸（宽, 高）	figsize=(8, 6) → 宽8英寸，高6英寸
dpi	分辨率（每英寸点数）	dpi=100
facecolor	背景色	facecolor='lightgray'

```
import matplotlib.pyplot as plt
```

```
# 创建一个宽10英寸、高6英寸、分辨率为100的画布，标题为"测试图表"
```

```
plt.figure(num="测试图表", figsize=(10, 6), dpi=100)
```

2. plt.plot() — 绘制图形

- plt.plot() 用于在当前的坐标轴上绘制折线图（也可以绘制点图、散点连线图等）。这是最核心的绘图命令。

基本语法：
plt.plot(x, y, fmt, ...)

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 50)
y = np.sin(x)

# 方式1：使用fmt快速格式
plt.plot(x, y, 'b--') # 蓝色虚线

# 方式2：使用关键字参数（更清晰）
plt.plot(x, y, color='red', linestyle='-', linewidth=2, marker='o', markersize=4, label='sin(x)')
```

参数	说明	示例
x	X轴数据（列表或数组）	[1, 2, 3, 4]
y	Y轴数据（列表或数组）	[1, 4, 9, 16]
fmt	快速格式字符串： [marker][line][color]	'ro--' → 红色圆点标记 + 虚线
color	线条颜色	color='red' 或 color='#FF0000'
linestyle	线型	linestyle='--'（虚线）、'-. '（点划线）
linewidth	线宽	linewidth=2
marker	数据点标记样式	marker='o'（圆点）、'^'（三角形）
label	图例标签	label="股价走势"

3. plt.show() — 显示图形

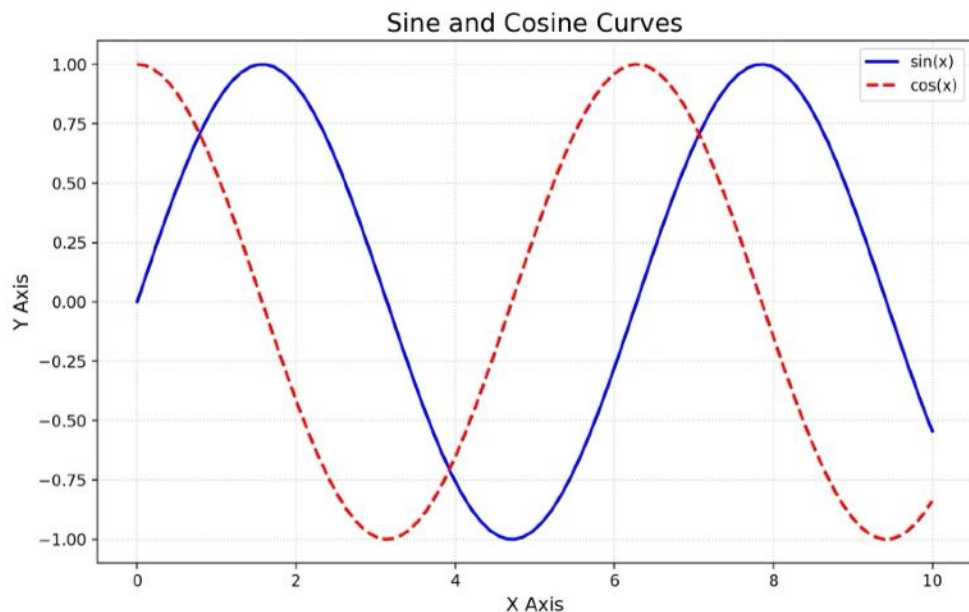
- plt.show() 用于显示当前所有已绘制的图形
- 重要说明
 - 必须调用：如果不调用 plt.show(), 在脚本模式下可能看不到任何图形输出
 - 阻塞/非阻塞：
 - 在脚本中, plt.show() 默认会阻塞程序执行, 直到关闭图形窗口。
 - 在 Jupyter Notebook 中, 它会自动显示图形, 通常不需要显式调用。

基本语法:
plt.show()

```
plt.plot([1, 2, 3], [1, 4, 9])  
plt.show() # 显示图形
```

三个函数的完整示例

函数	核心作用	必须调用?
plt.figure()	创建/指定画布	可选（不调用则用默认画布）
plt.plot()	绘制数据曲线	是（否则没内容可显示）
plt.show()	渲染并显示图形	是（脚本模式下必须）



```
import matplotlib.pyplot as plt
import numpy as np
```

```
# 1. Prepare data
```

```
x = np.linspace(0, 10, 100)
```

```
y1 = np.sin(x)
```

```
y2 = np.cos(x)
```

```
# 2. Create figure
```

```
plt.figure(figsize=(10, 6), dpi=100, facecolor='white')
```

```
# 3. Plot data
```

```
plt.plot(x, y1, color='blue', linestyle='-', linewidth=2, label='sin(x)')
```

```
plt.plot(x, y2, color='red', linestyle='--', linewidth=2, label='cos(x)')
```

```
# 4. Add chart elements
```

```
plt.title("Sine and Cosine Curves", fontsize=16)
```

```
plt.xlabel("X Axis", fontsize=12)
```

```
plt.ylabel("Y Axis", fontsize=12)
```

```
plt.legend(loc='upper right')
```

```
plt.grid(True, linestyle=':', alpha=0.6)
```

```
# 5. Display the plot
```

```
plt.show()
```



13.2

基本图表元素

概述

- 在 Matplotlib 中，整个图像为一个 Figure 对象。

在 Figure 对象中可以包含一个或者多个 Axes 对象。每个 Axes 对象都是一个拥有坐标系统的绘图区域。

基本图表元素的关系如图所示。



坐标轴

- 坐标轴是数据可视化的一个重要布局。

一个图形可包含任意数量的轴，但至少有一个，每个轴都可以设置一个轴标签。常见的是**二维坐标轴**，即包含 X 轴（横轴）和 Y 轴（纵轴），轴上可以**设置刻度标签**，使得生成的图像可以体现具体的数据值和相对位置。

- 使用 Matplotlib 画坐标图，往往需要对坐标轴设置很多参数，包括**横纵坐标轴范围、坐标轴刻度大小、坐标轴名称**等。Matplotlib 包含了很多函数，用来设置参数，如表所示。

方法	说明
<code>plt.xlim()</code> 、 <code>plt.ylim()</code>	设置横纵坐标轴范围
<code>plt.xlabel()</code> 、 <code>plt.ylabel()</code>	设置坐标轴名称
<code>plt.xticks()</code> 、 <code>plt.yticks()</code>	设置坐标轴刻度

1. plt.xlim() / plt.ylim() — 设置坐标轴范围

- 用于手动指定坐标轴的显示范围（最小值和最大值）

基本语法：

```
plt.xlim(left, right) # 设置 X 轴范围  
plt.ylim(bottom, top) # 设置 Y 轴范围
```

参数	说明
left / bottom	左边界/下边界（最小值）
right / top	右边界/上边界（最大值）

```
import matplotlib.pyplot as plt  
import numpy as np  
  
x = np.linspace(-5, 5, 100)  
y = x ** 2  
  
plt.plot(x, y)  
  
# 设置坐标轴范围  
plt.xlim(-3, 3) # X 轴显示 -3 到 3  
plt.ylim(0, 10) # Y 轴显示 0 到 10  
  
plt.show()
```

2. plt.xlabel() / plt.ylabel() — 设置坐标轴标签

- 为坐标轴添加描述性文字标签

基本语法:

```
plt.xlabel(label, fontsize=None, color=None, loc=None)
```

```
plt.ylabel(label, fontsize=None, color=None, loc=None)
```

参数	说明	示例
label	标签文本（字符串）	"时间 (秒)"
fontsize	字体大小	fontsize=14
color	字体颜色	color='red'
loc	标签位置	loc='left' / 'center' / 'right'

```
plt.plot([1, 2, 3], [1, 4, 9])
```

```
plt.xlabel("X Axis", fontsize=12, color='blue')
```

```
plt.ylabel("Y Axis", fontsize=12, color='blue')
```

```
plt.show()
```

3. plt.xticks() / plt.yticks() — 设置坐标轴刻度

- 用于自定义刻度位置和刻度标签

基本语法:

只设置刻度位置

```
plt.xticks(ticks)
```

同时设置刻度和标签

```
plt.xticks(ticks, labels, rotation=None, fontsize=None)
```

参数	说明	示例
ticks	刻度位置列表	[0, 1, 2, 3]
labels	刻度标签列表	['零', '一', '二', '三']
rotation	标签旋转角度	rotation=45
fontsize	标签字体大小	fontsize=10

```
plt.plot([1, 2, 3, 4], [1, 4, 9, 16])
```

X轴: 将 1,2,3,4 显示为一,二,三,四

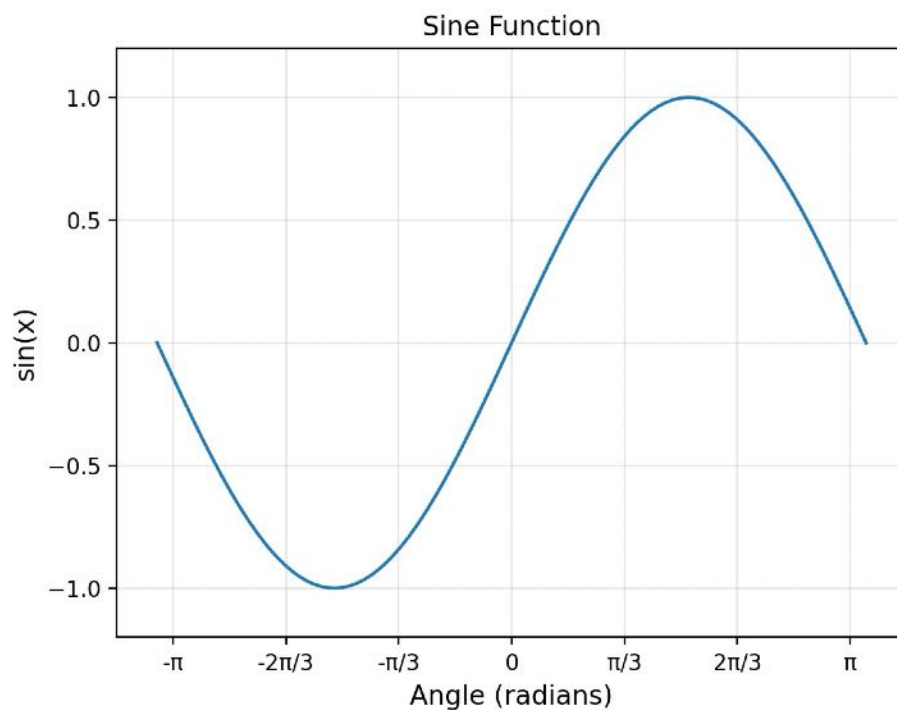
```
plt.xticks([1, 2, 3, 4], ['One', 'Two', 'Three', 'Four'])
```

Y轴: 指定刻度和标签

```
plt.yticks([0, 5, 10, 15, 20], ['0', '5', '10', '15', '20'])
```

```
plt.show()
```


完整示例



```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(-np.pi, np.pi, 100)
y = np.sin(x)
```

```
plt.plot(x, y)
```

```
# 设置范围
```

```
plt.xlim(-3.5, 3.5)
```

```
plt.ylim(-1.2, 1.2)
```

```
# 设置标签
```

```
plt.xlabel("Angle (radians)", fontsize=12)
```

```
plt.ylabel("sin(x)", fontsize=12)
```

```
# 设置刻度
```

```
plt.xticks([-3, -2, -1, 0, 1, 2, 3],
           ['-π', '-2π/3', '-π/3', '0', 'π/3', '2π/3', 'π'])
```

```
plt.title("Sine Function")
```

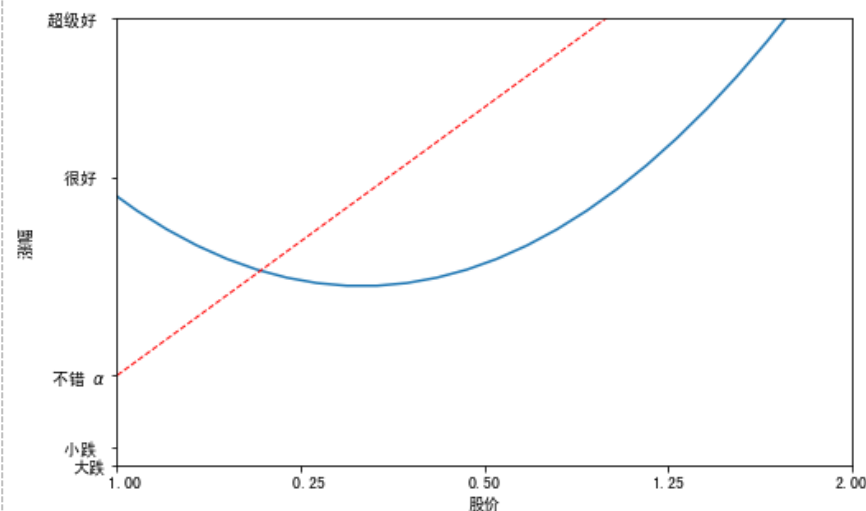
```
plt.grid(True, alpha=0.3)
```

```
plt.show()
```

案例1

【例】坐标轴设置。

```
① import matplotlib.pyplot as plt
   import numpy as np
② x = np.linspace(-3,3,50)
   y1 = 2*x + 1
   y2 = x**2
③ plt.figure(num=3,figsize=(8,5))
④ plt.plot(x,y2)
   plt.plot(x,y1,color='red',linewidth=1.0,linestyle='--')
⑤ plt.xlim((-1,2))
   plt.ylim((-2,3))
⑥ plt.xlabel(u'股价',fontproperties='SimHei')
   plt.ylabel(u'涨幅',fontproperties='SimHei')
   new_ticks = np.linspace(-1,2,5)
⑦ plt.xticks(new_ticks)
⑧ plt.yticks([-2,-1.8,-1,1.22,3.],
              ['大跌', '小跌', '不错  $\alpha$ ', '很好', '超级好'],
              fontproperties='SimHei')
⑨ plt.show()
```



图例

- 图例 Legend 是可视化图像中帮助识别图线的重要元素，尤其是图像中线型复杂、数量较多时。

Matplotlib中，设置图例有两种方式：

一种是添加 label 参数，
另一种是调用 `ax.legend()` 或 `plt.legend()`。

在 `plt.legend()` 中，可通过传入 loc 参数来调整图例的位置，具体的 loc 参数位置对应如表所示。

loc参数	对应数字
best	0
upper right	1
upper left	2
lower left	3
lower right	4
right	5
center left	6
center right	7
lower center	8
upper center	9
center	10

1. label 参数 — 为曲线添加标识

- label 是 plot() 函数中使用的参数，用于为每条曲线指定一个名称，这个名称随后会显示在图例中

基本语法：

```
plt.plot(x, y, label='曲线名称')
```

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)

# 为每条曲线添加 label
plt.plot(x, np.sin(x), label='sin(x)')
plt.plot(x, np.cos(x), label='cos(x)')
plt.plot(x, x**2, label='x^2')

# 必须调用 legend() 才会显示
plt.legend()
plt.show()
```

2. plt.legend() — 自动创建图例

- plt.legend() 是 Matplotlib 中最常用的图例命令，它会自动收集所有带有 label 的曲线，生成图例

基本语法：

```
plt.legend()
```

```
plt.legend(loc='位置')
```

```
plt.legend(handles, labels, loc='位置')
```

参数	说明	可选值
loc	图例位置	'best', 'upper right', 'lower left' 等
fontsize	字体大小	'small', 'medium', 'large', 或数值
ncol	列数	ncol=2（将图例排成2列）
title	图例标题	title='Legend'
frameon	是否显示边框	True / False

示例 1：基本用法

```
plt.plot(x, y1, label='Line A')
```

```
plt.plot(x, y2, label='Line B')
```

```
plt.legend() # 自动显示图例
```

示例 2：指定位置

```
plt.legend(loc='upper left') # 左上角
```

```
plt.legend(loc='lower right') # 右下角
```

```
plt.legend(loc='best') # 自动选择最佳位置
```

示例 3：自定义外观

```
plt.legend(loc='upper right', fontsize=12, ncol=2, title='Functions', frameon=True)
```

3. ax.legend() — 面向对象风格的图例

- 当使用面向对象方式绘图时（显式创建 Figure 和 Axes 对象），用 `ax.legend()` 而不是 `plt.legend()`。

基本语法：

```
fig, ax = plt.subplots()
ax.plot(x, y, label='Data')
ax.legend()
```

特性	plt.legend()	ax.legend()
使用场景	Pyplot 状态机风格	面向对象风格
作用对象	当前活动的坐标轴	指定的 Axes 对象
多子图时	只影响当前子图	可分别设置不同子图的图例

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(x, np.sin(x), label='sin(x)')
ax1.plot(x, np.cos(x), label='cos(x)')
ax1.legend(loc='upper right')
ax1.set_title('Trigonometric Functions')

ax2.plot(x, x, label='y = x')
ax2.plot(x, x**2, label='y = x²')
ax2.legend(loc='upper left')
ax2.set_title('Polynomial Functions')

plt.show()
```

Pyplot 风格 vs 面向对象风格的区别

特性	Pyplot 风格 (plt.legend())	面向对象风格 (ax.legend())
编程范式	状态机风格（类似 MATLAB）	面向对象风格
控制方式	隐式操作“当前”坐标轴	显式操作指定的 Axes 对象
代码简洁度	更简洁，适合快速绘图	稍复杂，但更清晰
多子图场景	容易混淆，需要用 plt.sca() 切换	天然支持，每个 ax 独立控制
灵活性	较低	更高，可精确控制每个元素
推荐场景	简单图表、数据分析快速探索	复杂图表、论文插图、多子图布局

操作	Pyplot 风格	面向对象风格
创建图形	plt.figure()	fig = plt.figure() 或 fig, ax = plt.subplots()
绘图	plt.plot(x, y)	ax.plot(x, y)
设置标题	plt.title("text")	ax.set_title("text")
设置 X 标签	plt.xlabel("text")	ax.set_xlabel("text")
设置 Y 标签	plt.ylabel("text")	ax.set_ylabel("text")
设置 X 范围	plt.xlim(0, 10)	ax.set_xlim(0, 10)
设置 Y 范围	plt.ylim(0, 10)	ax.set_ylim(0, 10)
添加图例	plt.legend()	ax.legend()
显示网格	plt.grid(True)	ax.grid(True)

```
import matplotlib.pyplot as plt
import numpy as np
```

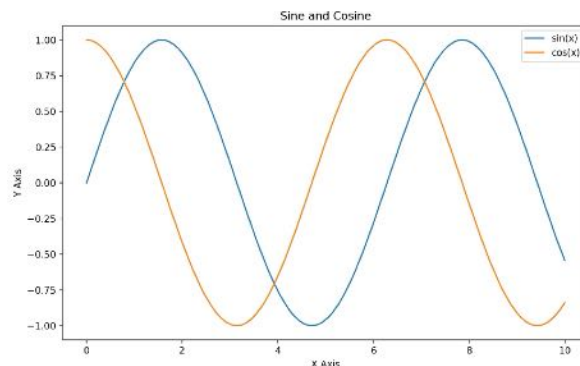
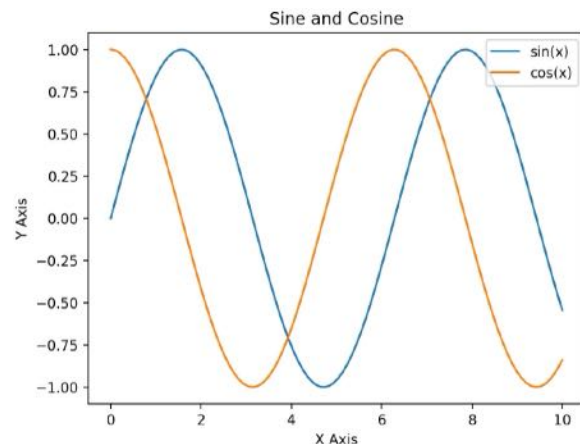
```
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
```

```
# 隐式创建 figure 和 axes
plt.plot(x, y1, label='sin(x)')
plt.plot(x, y2, label='cos(x)')
```

```
# 图例自动添加到“当前”axes
plt.legend(loc='upper right')
```

```
plt.title("Sine and Cosine")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```

Pyplot 风格 状态机模式



面向对象风格 OO 模式

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
```

```
# 显式创建 figure 和 axes
fig, ax = plt.subplots(figsize=(10, 6))
```

```
# 使用 ax 的方法绘图
ax.plot(x, y1, label='sin(x)')
ax.plot(x, y2, label='cos(x)')
```

```
# 图例直接添加到对应的 axes 对象
ax.legend(loc='upper right')
```

```
ax.set_title("Sine and Cosine")
ax.set_xlabel("X Axis")
ax.set_ylabel("Y Axis")
```

```
plt.show()
```


Q: 上一页的面向对象风格OO 模式代码, fig定义是否冗余?

```
fig, ax = plt.subplots(figsize=(10, 6))
```

- 这行代码同时创建了两个对象:
 - fig: Figure 对象 (画布)
 - ax: Axes 对象 (坐标轴)
- 可以省略 (使用 _ 占位符)
 - _ , ax = plt.subplots(figsize=(10, 6))
- 当前代码没用到 fig, 但保留有好处:
未来可扩展功能

```
fig, ax = plt.subplots(figsize=(10, 6))

# 后续可能需要使用 fig 的方法
fig.suptitle("Main Title")      # 添加总标题
fig.tight_layout()              # 自动调整布局
fig.savefig("plot.png", dpi=300) # 保存图片
fig.set_facecolor('lightgray')  # 设置背景色
```

Pyplot 风格

```
import matplotlib.pyplot as plt
import numpy as np
```

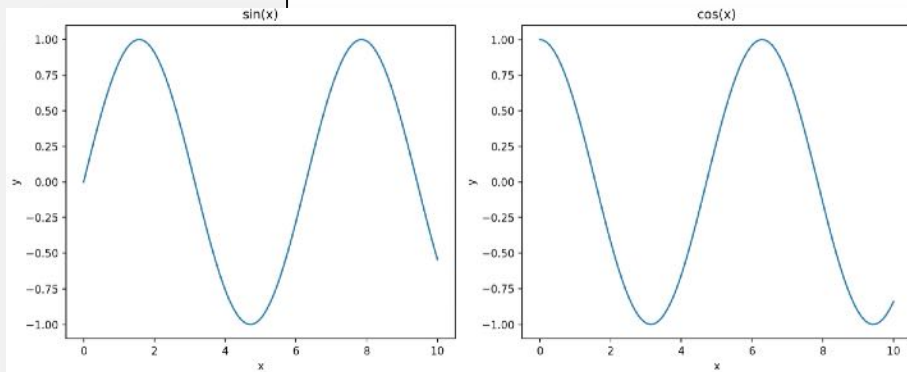
```
x = np.linspace(0, 10, 100)
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
plt.plot(x, np.sin(x))
plt.title("sin(x)")
plt.xlabel("x")
plt.ylabel("y")
```

```
plt.subplot(1, 2, 2)
plt.plot(x, np.cos(x))
plt.title("cos(x)")
plt.xlabel("x")
plt.ylabel("y")
```

```
plt.tight_layout()
plt.show()
```



多子图

以左右排列两个子图为例

面向对象风格

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(0, 10, 100)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
```

```
ax1.plot(x, np.sin(x))
ax2.plot(x, np.cos(x))
```

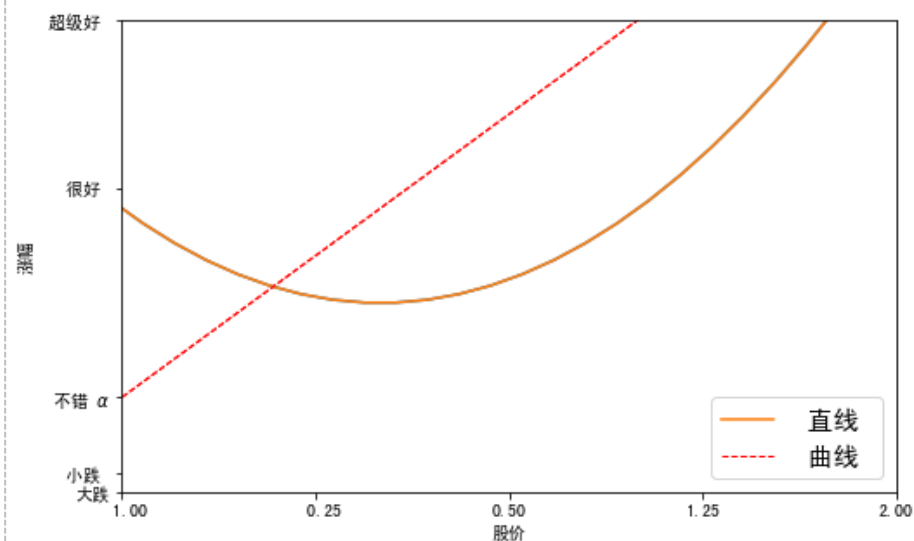
```
ax1.set(title="sin(x)", xlabel="x", ylabel="y")
ax2.set(title="cos(x)", xlabel="x", ylabel="y")
```

```
plt.tight_layout()
plt.show()
```

案例2

【例】 增加图例： 在前例的基础上增加描述图线类型的图例。

```
❶ import matplotlib.pyplot as plt
import numpy as np
❷ x = np.linspace(-3,3,50)
y1 = 2*x + 1
y2 = x**2
❸ plt.figure(num=3,figsize=(8,5))
❹ plt.plot(x,y2)
plt.plot(x,y1,color='red',linewidth=1.0,linestyle='--')
plt.xlim((-1,2))
plt.ylim((-2,3))
plt.xlabel(u'股价',fontproperties='SimHei')
plt.ylabel(u'涨幅',fontproperties='SimHei')
❺ new_ticks = np.linspace(-1,2,5)
plt.xticks(new_ticks)
plt.yticks([-2,-1.8,-1,1.22,3.],
            ['大跌', '小跌', '不错  $\alpha$ ', '很好', '超级好'],
            fontproperties='SimHei')
❻ l1=plt.plot(x,y2)
❼ l2=plt.plot(x,y1,color='red',linewidth=1.0,linestyle='--')
❽ plt.legend(handles=[l1,l2],
              prop={'family':'SimHei','size':15},
              loc='lower right',labels=[' 直线 ', ' 曲线 '])
❾ plt.show()
```



标注

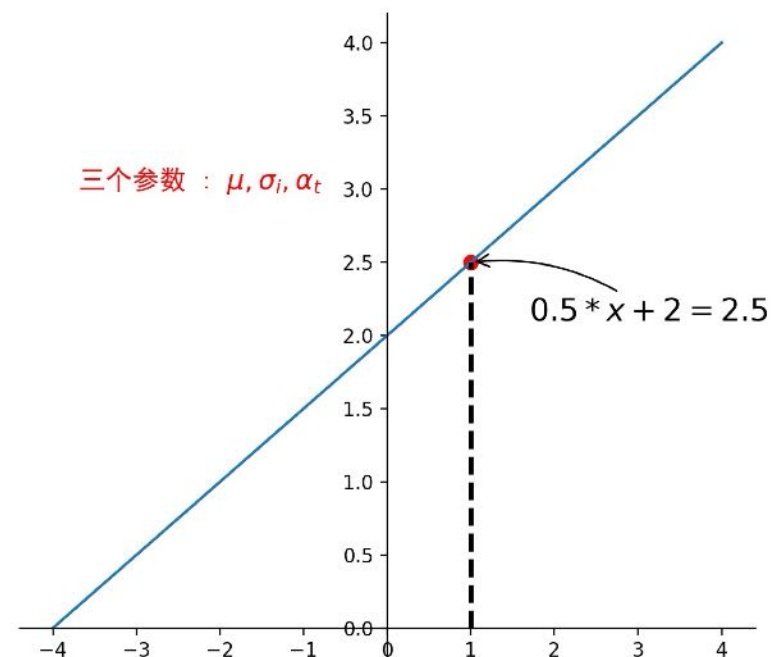
图形上给数据添加文本注解能够帮助读者快速理解图像和定位数据，方便我们在合适的位置添加描述信息。

annotate() 支持带箭头的划线工具。
text() 是一种无指向型注释。
调用语句和参数说明如表所示。

注释函数	参数	含义
annotate()	s	str, 注释信息内容
	xy	(float,float), 箭头点所在的坐标位置
	xytext	(float,float), 注释内容的坐标位置
	weight	str or int, 设置字体线型。其中字符串从小到大可选项有{'ultralight', 'light', 'normal', 'regular', 'book', 'medium', 'roman', 'semibold', 'demibold', 'demi', 'bold', 'heavy', 'extra bold', 'black'}
	color	str or tuple, 设置字体颜色, 单个字符候选项{'b', 'g', 'r', 'c', 'm', 'y', 'k', 'w'}, 也可以'black', 'red'等, tuple 时用 [0,1]之间的浮点型数据, RGB 或者 RGBA, 如: (0.1, 0.2, 0.5)、(0.1, 0.2, 0.5, 0.3)等
	arrowprops	dict, 设置指向箭头的参数, 字典中 key 值有 ①arrowstyle: 设置箭头的样式 ②connectionstyle: 设置箭头的形状, 为直线或者曲线 ③color: 设置箭头颜色, 见前面的 color 参数
text()	bbox	dict, 为注释文本添加边框
	x, y	代表注释内容位置
	s	代表注释文本内容
	family	设置字体
	fontsize	字体大小
	style	设置字体样式

案例3：增加标注

```
import matplotlib.pyplot as plt
import numpy as np
❶ x=np.linspace(-4,4,20)
  y=0.5*x+2
plt.figure(num=1,figsize=(6,6))
❷ plt.plot(x,y)
❸ ax=plt.gca()
❹ ax.spines['right'].set_color('none')
  ax.spines['top'].set_color('none')
  ax.xaxis.set_ticks_position('bottom')
  ax.spines['bottom'].set_position(('data',-0))
  ax.yaxis.set_ticks_position('left')
  ax.spines['left'].set_position(('data',0))
  x0=1
  y0=0.5*x0+2
❺ plt.scatter(x0,y0,s=50,color='r')
❻ plt.plot([x0,x0],[0,y0],'k--',lw=2.5)
❼ plt.annotate(r'$0.5*x+2=%s$'%y0, xy=(x0,y0), xycoords='data',
               xytext=(+30,-30), textcoords='offset points',
               fontsize=16,
               arrowprops=dict(arrowstyle='->',
                               connectionstyle='arc3,rad=.2'))
❽ plt.text(-3.7,3,'三个参数 : $\mu,\sigma_i,\alpha_t$',
           fontdict={'size':'14','color':'red'},
           fontproperties='SimHei')
❾ plt.show()
```



1. text() — 添加普通文字

基础语法:

```
plt.text(x, y, text, fontsize=12, color='black', ha='center', va='center')
```

```
ax.text(x, y, text, fontsize=12, color='black')
```

参数	说明	示例
x, y	文字位置坐标	x=2, y=0.5
text	要显示的文字	'峰值点'
fontsize	字体大小	fontsize=14
color	文字颜色	color='red'
ha	水平对齐: 'left', 'center', 'right'	ha='center'
va	垂直对齐: 'top', 'center', 'bottom'	va='bottom'
rotation	旋转角度	rotation=45

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
x = np.linspace(0, 10, 100)
```

```
y = np.sin(x)
```

```
plt.plot(x, y)
```

```
# 在坐标 (5, 0) 处添加文字
```

```
plt.text(5, 0.1, 'sin(x) 曲线', fontsize=12, ha='center')
```

```
# 在峰值附近添加文字
```

```
plt.text(1.57, 0.95, '最大值', fontsize=10, color='red', ha='center')
```

```
plt.show()
```

2. annotate() — 添加带箭头的标注

基础语法:

```
plt.annotate(text, xy, xytext=None, arrowprops=None, ...)
```

```
ax.annotate(text, xy, xytext=None, arrowprops=None, ...)
```

参数	说明	示例
text	标注文字	'局部最大值'
xy	箭头指向的点（被标注的位置）	xy=(1.57, 1)
xytext	文字放置的位置 （默认与 xy 相同）	xytext=(3, 0.8)
arrowprops	箭头属性字典	arrowprops=dict(facecolor='red', shrink=0.05)
fontsize	字体大小	fontsize=12
ha / va	文字对齐方式	ha='center'

arrowprops 常用设置:

```
arrowprops=dict(  
    facecolor='red',    # 箭头颜色  
    edgecolor='black',  # 边框颜色  
    width=1,            # 箭头宽度  
    headwidth=8,        # 箭头头部宽度  
    headlength=8,       # 箭头头部长度  
    shrink=0.05,        # 箭头两端收缩比例  
    arrowstyle='->'     # 箭头样式  
)
```

```
import numpy as np
```

```
x = np.linspace(0, 10, 100)
```

```
y = np.sin(x)
```

```
plt.plot(x, y)
```

```
# 标注最高点
```

```
plt.annotate('局部最大值',
```

```
            xy=(1.57, 1),    # 箭头指向的位置
```

```
            xytext=(3, 0.8), # 文字放置的位置
```

```
            arrowprops=dict(facecolor='red', shrink=0.05),
```

```
            fontsize=12)
```

```
plt.show()
```

能见度

- 图片里的内容过多时，可能会造成一些信息的覆盖。为了方便读取完整的数据内容，需要设置标签相关的能见度，以确保部分信息不会被重叠覆盖。可使用 bbox 参数设置标签属性。

```
label.set_bbox(dict(facecolor='white',edgecolor='none',alpha=0.7))
```

其中，facecolor 代表标签颜色，edgecolor 表示标签边缘颜色，alpha 是透明度属性。

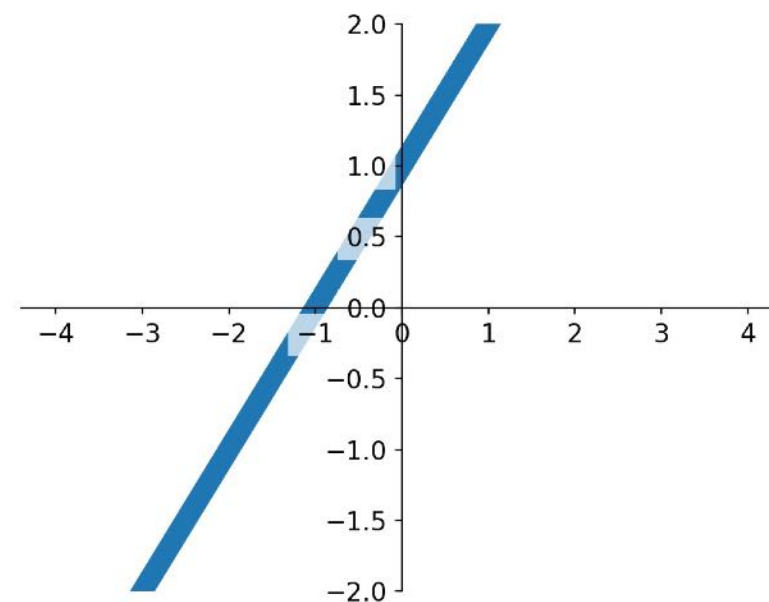
- 在设置能见度的过程中，常遇到标签不能显示的问题。可以设置 zorder，让标签显示于图像之上。

```
plt.plot(x,y,linewidth=10,zorder=1)  
label.set_zorder(100)
```


案例4

【例】 能见度设置：在坐标轴中设置可能会被遮挡的内容的透明度。

```
❶ import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-4,4,20)
❷ y = x + 1
plt.figure()
plt.plot(x, y, linewidth=10, zorder=1)
❸ plt.ylim(-2, 2)
❹ ax = plt.gca()
ax.spines['right'].set_color('none')
❺ ax.spines['top'].set_color('none')
ax.xaxis.set_ticks_position('bottom')
ax.spines['bottom'].set_position(('data', 0))
ax.yaxis.set_ticks_position('left')
❻ ax.spines['left'].set_position(('data', 0))
❼ for label in ax.get_xticklabels()+ax.get_yticklabels():
    label.set_fontsize(12)
    label.set_bbox(dict(facecolor='white',
                        edgecolor='none',alpha=0.7))
❽ plt.show()
```



1. bbox — 文字背景框（边界框）

- bbox 用于给文字添加一个背景框，让文字更醒目

基本语法：

```
plt.text(x, y, text, bbox=dict(参数))
```

```
plt.annotate(text, xy=(x, y), bbox=dict(参数))
```

参数	说明	示例
boxstyle	框的形状	'round'(圆角), 'square'(方形), 'round4'
facecolor	背景色	facecolor='yellow'
edgecolor	边框颜色	edgecolor='red'
alpha	透明度 (0-1)	alpha=0.7
pad	内边距	pad=0.5

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
x = np.linspace(0, 10, 100)
```

```
y = np.sin(x)
```

```
plt.plot(x, y)
```

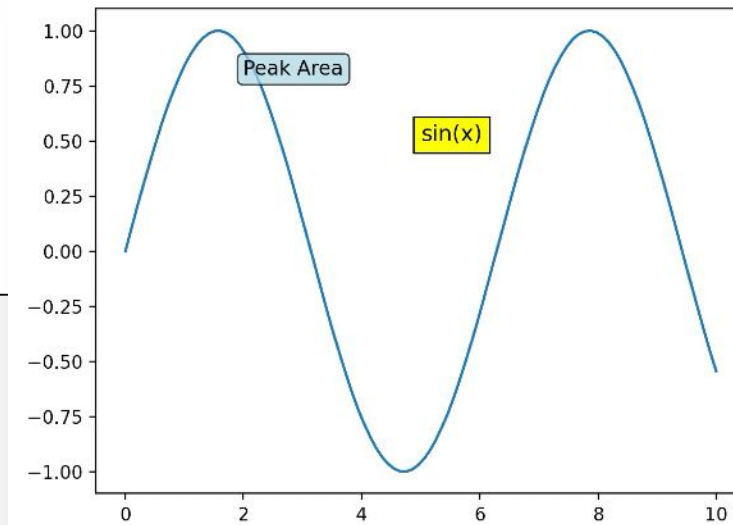
```
# 简单背景框
```

```
plt.text(5, 0.5, 'sin(x)', fontsize=12,  
        bbox=dict(facecolor='yellow', edgecolor='black'))
```

```
# 圆角框 + 透明度
```

```
plt.text(2, 0.8, 'Peak Area', fontsize=11,  
        bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.7))
```

```
plt.show()
```



2. zorder — 图层顺序

- zorder 控制元素的**前后堆叠顺序**
- 数值越大，显示在越上层

基本语法:

```
plt.plot(x, y, zorder=数值)
```

```
ax.plot(x, y, zorder=数值)
```

元素	默认 zorder
网格线 (grid)	1.5
曲线 (plot)	2
散点 (scatter)	2
柱状图 (bar)	2
填充区域 (fill_between)	1
文字 (text / annotate)	3
图例 (legend)	5

默认值: 大部分元素 zorder=0

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(0, 10, 100)
```

```
y = np.sin(x)
```

```
plt.figure(figsize=(10, 6))
```

```
# 绘制曲线 (设置高 zorder)
```

```
plt.plot(x, y, 'b-', linewidth=3, label='sin(x)', zorder=3)
```

```
# 绘制散点 (更高 zorder, 显示在曲线上方)
```

```
plt.scatter([1.57, 4.71, 7.85], [1, -1, 1],
            color='red', s=100, zorder=4, label='Extreme Point')
```

```
# 添加带背景框的文字
```

```
plt.text(5, 0.6, 'y = sin(x)', fontsize=14,
        bbox=dict(boxstyle='round', facecolor='lightyellow', edgecolor='orange'),
        zorder=5)
```

```
# 网格线 (低 zorder, 最底层)
```

```
plt.grid(True, linestyle='--', alpha=0.5, zorder=1)
```

```
# 设置坐标轴范围
```

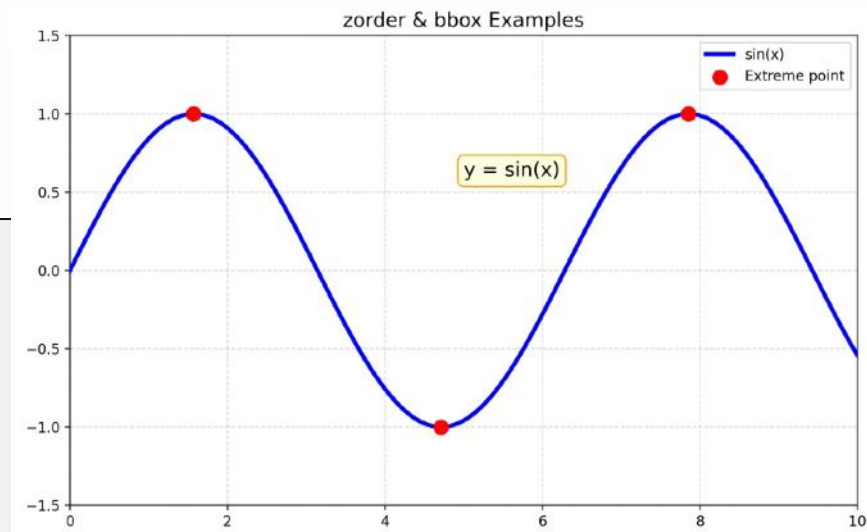
```
plt.xlim(0, 10)
```

```
plt.ylim(-1.5, 1.5)
```

```
plt.legend()
```

```
plt.title('zorder & bbox Examples', fontsize=14)
```

```
plt.show()
```



图表的保存

- 完成图表绘制后，可以让程序自动将图表保存到文件中。

可将`plt.show()` 的调用替换为 `plt.savefig()` 的调用。

```
plt.savefig('figure_plot.png',bbox_inches='tight')
```

其中，第一个参数指定要以什么样的文件名保存图表，这个文件将存储到执行程序所在的目录中。第二个参数指定将图表多余的空白区域裁减掉。



13.3

常见图形绘制

散点图

- 散点图经常用来显示分布或者比较几个变量的相关性或者分组。

绘制散点图的关键是 `scatter()` 函数。

```
plt.scatter(x, y, s, c, marker)
```

其中，`x`、`y`表示坐标点的位置，`s`代表点的大小/粗细标量，`c`表示点的颜色，`marker`是标记的样式，默认是 `'o'`。

散点图 scatter()

- scatter() 用于绘制散点图，每个点可以独立设置大小、颜色，比 plot() 更灵活

基本语法：

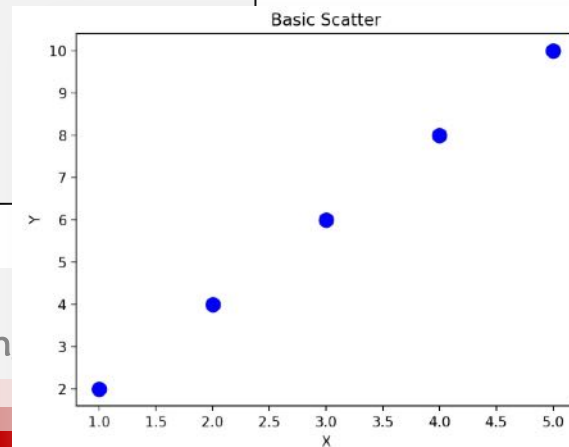
```
plt.scatter(x, y, s=None, c=None, marker=None, alpha=None, cmap=None, label=None, **kwargs)
```

参数	说明	示例
x, y	点的坐标	x=[1,2,3], y=[4,5,6]
s	点的大小 (像素 ²)	s=50 或 s=[20,50,100]
c	点的颜色	c='red' 或 c=[1,2,3]
marker	点的形状	marker='o'(圆), '^'(三角), 's'(方块)
alpha	透明度 (0-1)	alpha=0.5
cmap	颜色映射	cmap='viridis'
label	图例标签	label='数据组A'

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
```

```
plt.scatter(x, y, color='blue', s=100)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Basic Scatter')
plt.show()
```



散点图 scatter()

s — 控制点的大小

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.random.rand(50)
y = np.random.rand(50)
```

随机大小

```
sizes = np.random.randint(20, 200, 50)
```

```
plt.scatter(x, y, s=sizes, alpha=0.6)
plt.title('点大小不同')
plt.show()
```

c — 控制点的颜色

单一颜色

```
plt.scatter(x, y, c='red', s=50)
```

根据数值变化颜色

```
colors = np.random.rand(50)
plt.scatter(x, y, c=colors, cmap='viridis', s=50)
plt.colorbar() # 显示颜色条
plt.show()
```

marker — 点形状

```
markers = ['o', '^', 's', 'D', 'v', '*']
for i, m in enumerate(markers):
    x = np.random.rand(10)
    y = np.random.rand(10)
    plt.scatter(x + i, y, marker=m, label=m, s=80)
plt.legend()
plt.show()
```



```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# 生成三组数据
```

```
np.random.seed(42)
```

```
group1_x = np.random.normal(2, 0.5, 50)
```

```
group1_y = np.random.normal(3, 0.5, 50)
```

```
group2_x = np.random.normal(5, 0.5, 50)
```

```
group2_y = np.random.normal(4, 0.5, 50)
```

```
group3_x = np.random.normal(8, 0.5, 50)
```

```
group3_y = np.random.normal(2, 0.5, 50)
```

```
# 绘制散点图
```

```
plt.figure(figsize=(10, 6))
```

```
plt.scatter(group1_x, group1_y, c='blue', s=60, alpha=0.7, label='Group A')
```

```
plt.scatter(group2_x, group2_y, c='red', s=60, alpha=0.7, label='Group B')
```

```
plt.scatter(group3_x, group3_y, c='green', s=60, alpha=0.7, label='Group C')
```

```
plt.xlabel('X Axis', fontsize=12)
```

```
plt.ylabel('Y Axis', fontsize=12)
```

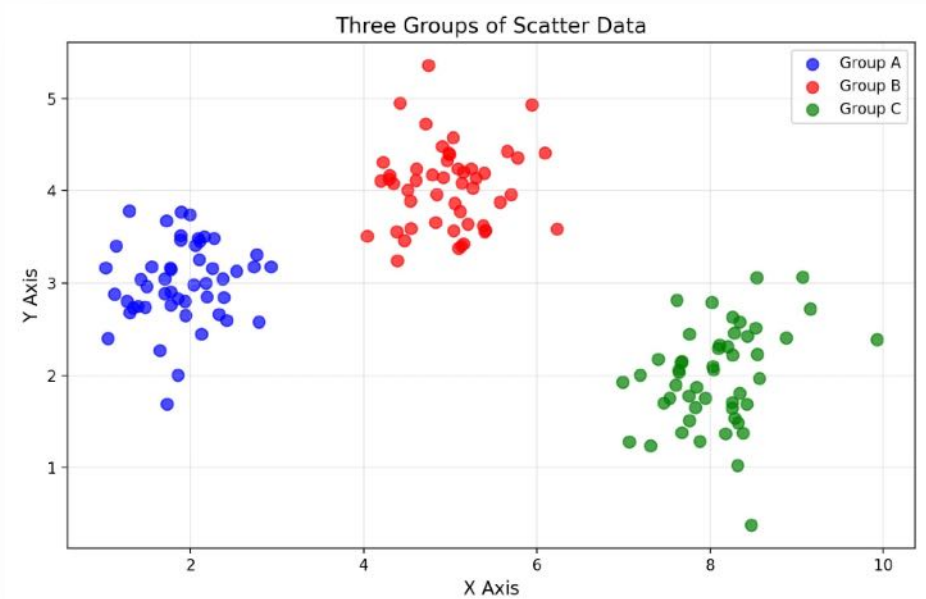
```
plt.title('Three Groups of Scatter Data', fontsize=14)
```

```
plt.legend()
```

```
plt.grid(True, alpha=0.3)
```

```
plt.show()
```

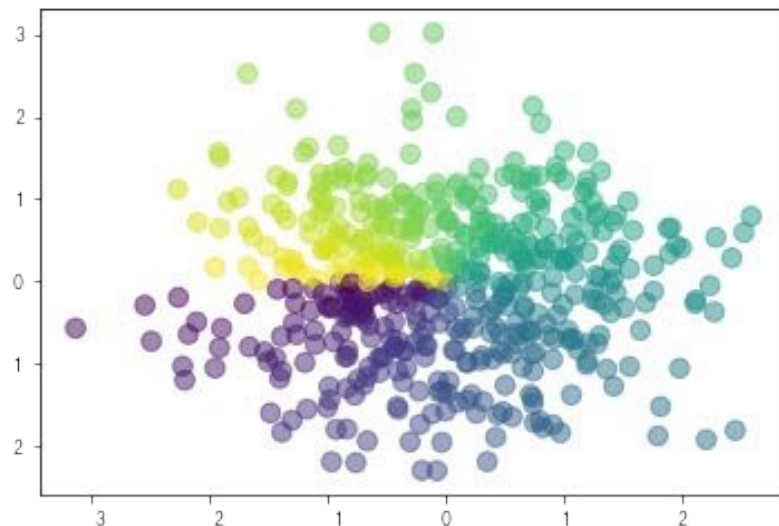
Scatter() 示例



案例5

【例】散点图示例。

```
① import matplotlib.pyplot as plt
② import numpy as np
③ n = 500
④ X = np.random.normal(0,1,n) #X坐标: 正态分布
⑤ Y = np.random.normal(0,1,n) #Y坐标: 正态分布
⑥ T = np.arctan2(Y,X)          #T= 每个点的角度 (- $\pi$  到  $\pi$ )
⑦ plt.scatter(X,Y,s=75,c=T,alpha=0.5) # c=T 将每个点的角度值映射为颜色
⑧ plt.show()
```



折线图

- 在 matplotlib 面向对象的绘图库中，pyplot 是一个方便常用的接口。在绘制折线图时，plot() 函数既能够创建单条折线的折线图，也可以支持多条折线的复合图。

调用方法也很简单，只要在 plot() 函数中传入对应的 X 轴和 Y 轴数据即可。

```
plt.plot(x, y, color, linewidth, linestyle)
```

其中，x、y 是对应的坐标数据，color 用来指定折线的颜色，linewidth 指定折线的宽度，linestyle 指定折线的样式。

折线图plt.plot()

- plt.plot() 是 Matplotlib 中最基础的绘图函数，用于绘制折线图

基本语法：
plt.plot(x, y, fmt, **kwargs)

fmt 格式字符串
格式：[marker][line][color]

标记 (marker)	线型 (line)	颜色 (color)
o 圆点	- 实线	r 红
^ 三角	-- 虚线	g 绿
s 方块	: 点线	b 蓝
* 星形	-. 点划线	c 青
+ 加号		m 紫
x 叉号		y 黄
. 小点		k 黑

参数	说明	示例
x	X轴数据	[1,2,3,4] 或数组
y	Y轴数据	[1,4,9,16] 或数组
fmt	格式字符串：[marker][line][color]	'ro--'
color	线条颜色	color='red', color='#FF0000'
linestyle	线型	linestyle='--', '-', ':', '-.'
linewidth	线宽	linewidth=2
marker	数据点标记	marker='o', '^', 's', '*'
markersize	标记大小	markersize=8
label	图例标签	label='sin(x)'
alpha	透明度	alpha=0.5

```
import matplotlib.pyplot as plt
import numpy as np

days = np.arange(1, 21)
prices = 100 + np.cumsum(np.random.randn(20) * 2)

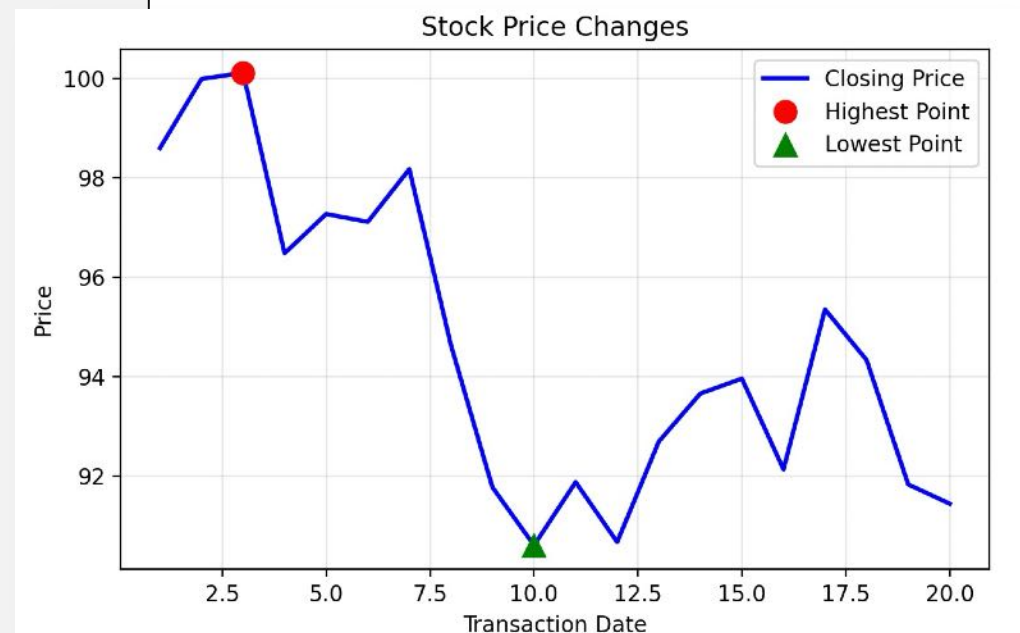
plt.figure(figsize=(12, 5))

# 主趋势线
plt.plot(days, prices, 'b-', linewidth=2, label='Closing Price')

# 标注最高点
max_idx = np.argmax(prices)
plt.plot(days[max_idx], prices[max_idx], 'ro', markersize=10, label='Highest Point')

# 标注最低点
min_idx = np.argmin(prices)
plt.plot(days[min_idx], prices[min_idx], 'g^', markersize=10, label='Lowest Point')

plt.xlabel('Transaction Date')
plt.ylabel('Price')
plt.title('Stock Price Changes')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

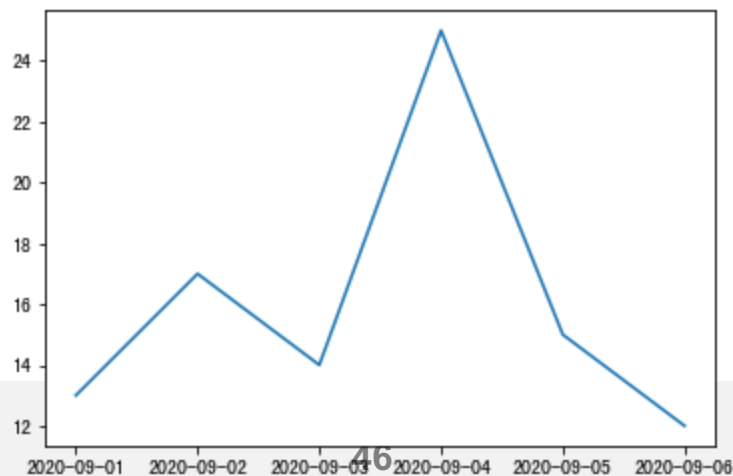


折线图示例：
股票走势

案例6

【例】折线图示例。

```
❶ import matplotlib.pyplot as plt
❷ import numpy as np
❸ import math
❹ import datetime
❺ recordings = [datetime.date(2020,9,1),datetime.date(2020,9,2),
               datetime.date(2020,9,3), datetime.date(2020,9,4),
               datetime.date(2020,9,5),datetime.date(2020,9,6)]
❻ words = [13,17,14,25,15,12]
❼ plt.plot(recordings,words)
❽ plt.show()
```



柱状图

- 柱状图常用于二维数据集，只需要比较一个维度。

普通柱状图：可反映一组数据内部时间变化或事物变化的某项指标的差异。

复合型多组柱状图：能比较多组数据集 各项间的变化。

柱状图利用柱状不同高度反映数据的差异，但不适用于 规模较大的数据集。

matplotlib 中使用 bar() 函数来绘制柱状图。

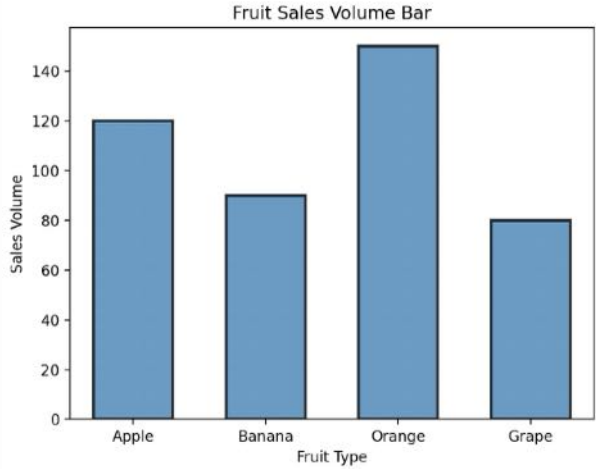
```
matplotlib.pyplot.bar(left, height, alpha, width, color, edgecolor,  
label, lw)
```

其中，left 表示 X 轴的位置序列，一般采用 range 函数产生一个序列，也可以是字符串；height 表示 Y 轴的数值序列，即柱形图的高度；alpha 度量的是透明度，一般值越小 越透明；width 描述了柱状图的宽度，一般为 0.8；color 或 facecolor 代表柱形图填充的颜色；edgecolor 用来设置图形边缘颜色；label 是解释每个柱子代表的含义，即 bar 的标签；linewidth 或 lw 用于调整边缘线的宽。

柱状图 plt.bar()

- 柱状图（条形图），适合展示分类数据的数值比较

基本语法：
plt.bar(x, height, width=0.8, bottom=None, align='center', **kwargs)



参数	说明	示例
x	柱子中心位置（类别）	['A','B','C'] 或 [1,2,3]
height	柱子的高度（数值）	[10,20,15]
width	柱子宽度	width=0.6（默认0.8）
bottom	底部起始高度	bottom=5（用于堆叠图）
color	柱子颜色	color='blue'
edgecolor	边框颜色	edgecolor='black'
label	图例标签	label='销量'
alpha	透明度	alpha=0.7

```
import matplotlib.pyplot as plt

categories = ['Apple', 'Banana', 'Orange', 'Grape']
sales = [120, 90, 150, 80]

plt.bar(categories, sales,
        width=0.6,      # 柱子宽度
        color='steelblue', # 填充色
        edgecolor='black', # 边框色
        linewidth=2,     # 边框宽度
        alpha=0.8)       # 透明度

plt.xlabel('Fruit Type')
plt.ylabel('Sales Volume')
plt.title('Fruit Sales Volume Bar')
plt.show()
```



```

import matplotlib.pyplot as plt
import numpy as np

# 数据
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales_2023 = [120, 135, 150, 170, 190, 210]
sales_2024 = [130, 148, 165, 185, 205, 230]
x = np.arange(len(months))
width = 0.35

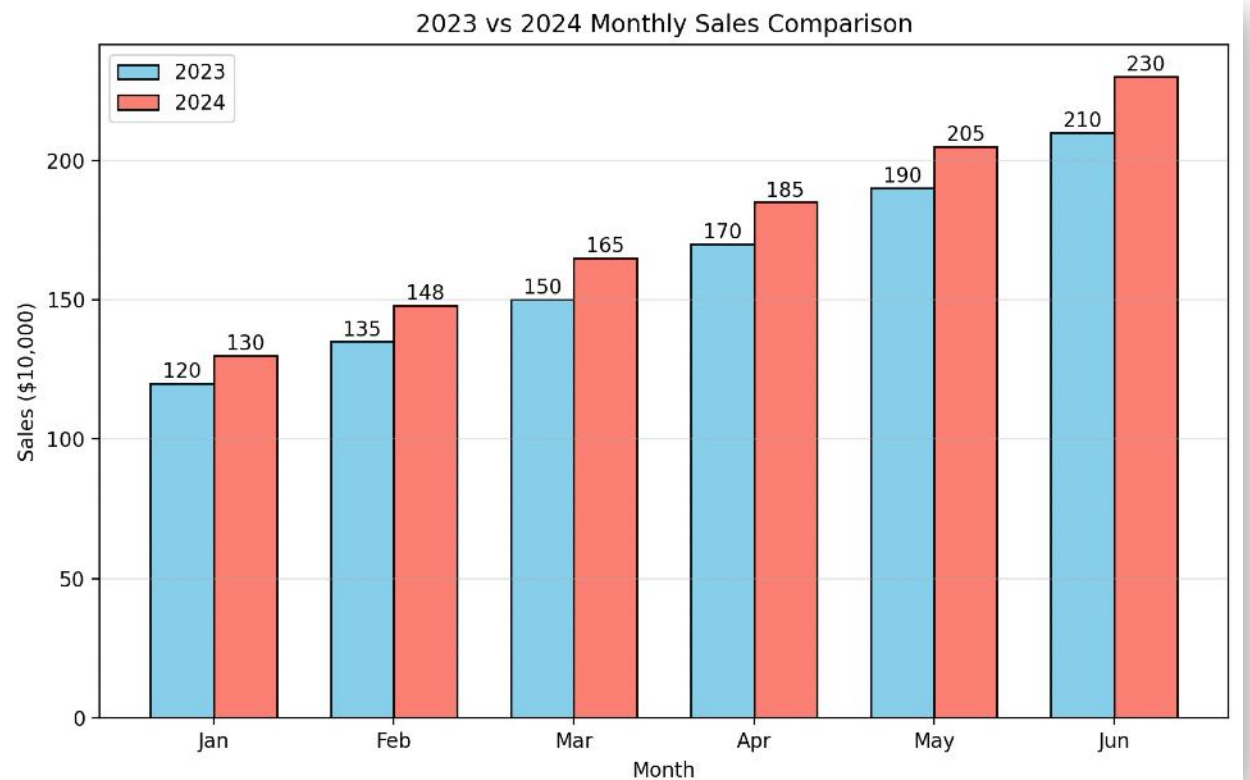
plt.figure(figsize=(10, 6))

# 绘制两组柱状图
bars1 = plt.bar(x - width/2, sales_2023, width,
                label='2023', color='skyblue', edgecolor='black')
bars2 = plt.bar(x + width/2, sales_2024, width,
                label='2024', color='salmon', edgecolor='black')

# 添加数值标签
for bar in bars1:
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 2,
             str(bar.get_height()), ha='center', fontsize=10)
for bar in bars2:
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 2,
             str(bar.get_height()), ha='center', fontsize=10)

plt.xticks(x, months)
plt.xlabel('Month')
plt.ylabel('Sales ($10,000)')
plt.title('2023 vs 2024 Monthly Sales Comparison')
plt.legend()
plt.grid(True, axis='y', alpha=0.3)
plt.show()

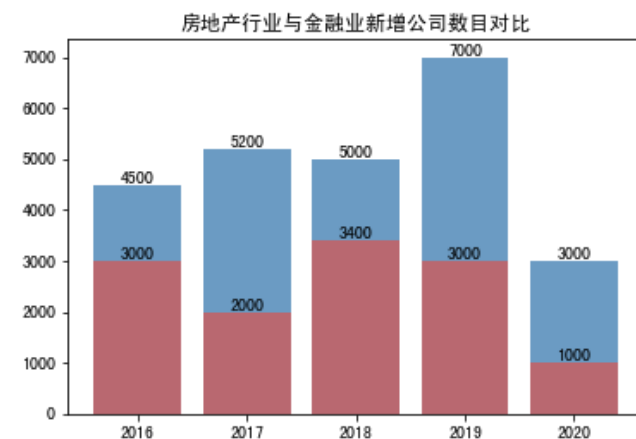
```



案例7

【例】柱状图示例。

```
① import matplotlib
   import matplotlib.pyplot as plt
② matplotlib.rcParams['font.sans-serif'] = ['SimHei']
③ x = ['2016', '2017', '2018', '2019', '2020']
④ y1 = [4500, 5200, 5000, 7000, 3000]
   y2 = [3000, 2000, 3400, 3000, 1000]
⑤ plt.bar(x=x, height=y1, label='房地产行业', color='steelblue', alpha=0.8)
   plt.bar(x=x, height=y2, label='金融业', color='indianred', alpha=0.8)
⑥ for x1, yy in zip(x, y1):
    plt.text(x1, yy + 1, str(yy), ha='center',
             va='bottom', fontsize=10, rotation=0)
⑦ for x1, yy in zip(x, y2):
    plt.text(x1, yy + 1, str(yy), ha='center',
             va='bottom', fontsize=10, rotation=0)
⑧ plt.title('房地产行业与金融业新增公司数目对比')
   plt.show()
```



subplot 子图

- 在一张图片中显示多个小图表进行对比，在 matplotlib 中称为子图。

可使用 subplot() 函数绘制子图。

```
plt.subplot(numbRow,numbCol,plotNum)
```

numbRow 表示 plot 图的行数，numbCol 是 plot 图的列数，plotNum 指 numbRow 与 numbCol 的矩阵中的第几幅图。

- 例如，subplot(2,2,3)中包含 4 个子图，构成 2*2 的矩阵图阵，3 则表示的是第 3 幅图。
- 如果是单数个子图，那么最后一幅图占据的空间可能较大，可以根据实际布局修改行数或者列数。
- 注意，subplot 函数中几个参数之间可以不用逗号连接，直接写成 subplot(223)。

最简单的subplot示例 (4个子图)

```
import matplotlib.pyplot as plt
```

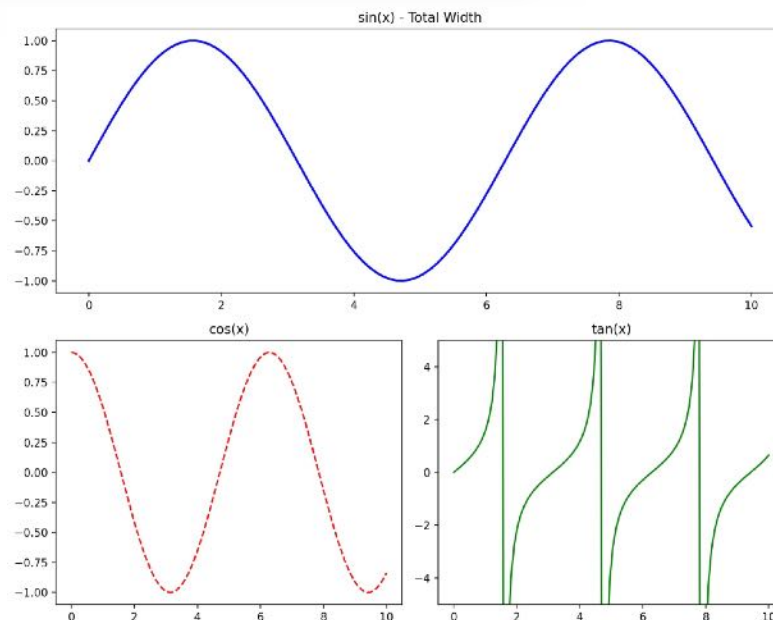
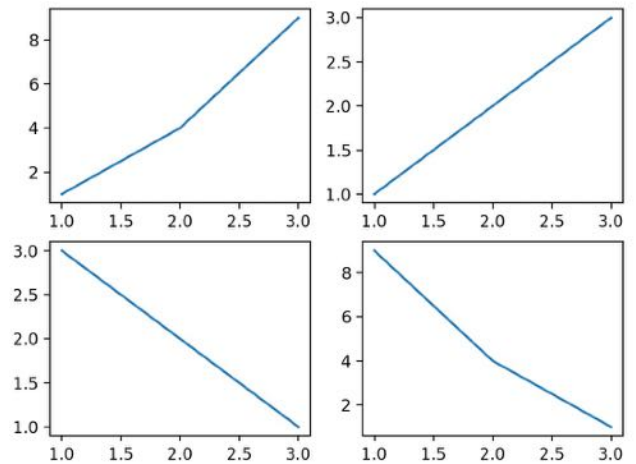
```
plt.subplot(2, 2, 1) # 第1个子图  
plt.plot([1, 2, 3], [1, 4, 9])
```

```
plt.subplot(2, 2, 2) # 第2个子图  
plt.plot([1, 2, 3], [1, 2, 3])
```

```
plt.subplot(2, 2, 3) # 第3个子图  
plt.plot([1, 2, 3], [3, 2, 1])
```

```
plt.subplot(2, 2, 4) # 第4个子图  
plt.plot([1, 2, 3], [9, 4, 1])
```

```
plt.show()
```



```
import matplotlib.pyplot as plt  
import numpy as np
```

```
x = np.linspace(0, 10, 100)
```

```
plt.figure(figsize=(10, 8))
```

第一行: 占满整行 (2列合并)

```
plt.subplot(2, 2, (1, 2)) # 第1行, 占1-2列  
plt.plot(x, np.sin(x), 'b-', linewidth=2)  
plt.title('sin(x) - 全宽')
```

第二行左

```
plt.subplot(2, 2, 3)  
plt.plot(x, np.cos(x), 'r--')  
plt.title('cos(x)')
```

第二行右

```
plt.subplot(2, 2, 4)  
plt.plot(x, np.tan(x), 'g-')  
plt.title('tan(x)')  
plt.ylim(-5, 5)
```

```
plt.tight_layout()  
plt.show()
```

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(0, 10, 100)
```

```
plt.figure(figsize=(10, 8))
```

```
# 子图1: 正弦
```

```
plt.subplot(2, 2, 1)
plt.plot(x, np.sin(x), 'b-')
plt.title('sin(x)')
plt.grid(True)
```

```
# 子图2: 余弦
```

```
plt.subplot(2, 2, 2)
plt.plot(x, np.cos(x), 'r--')
plt.title('cos(x)')
plt.grid(True)
```

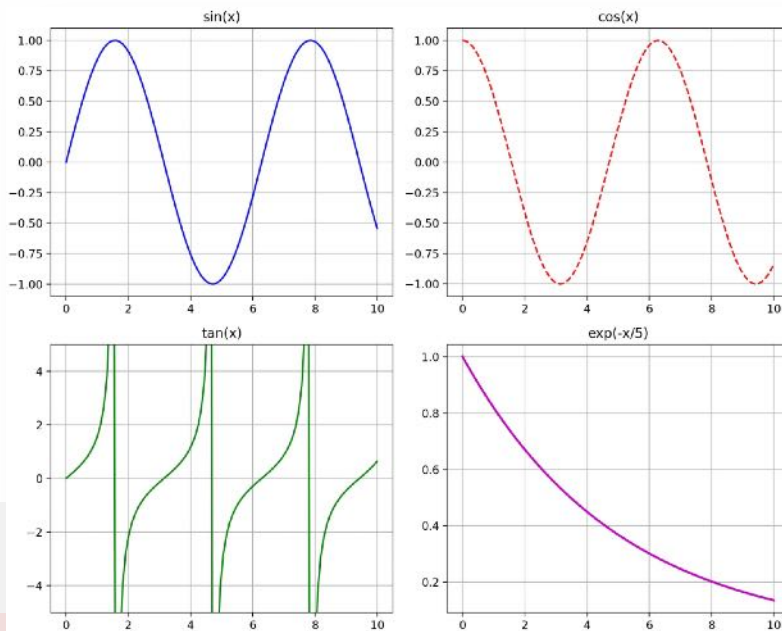
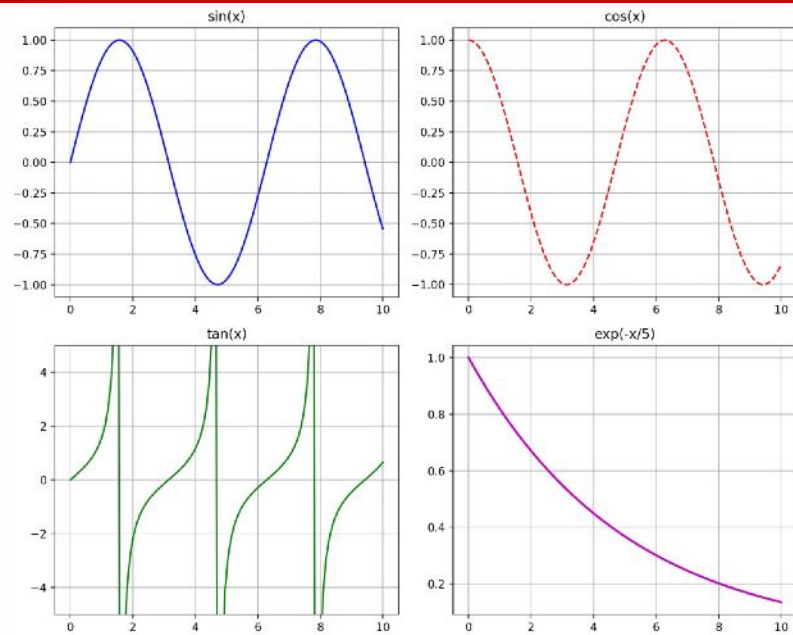
```
# 子图3: 正切
```

```
plt.subplot(2, 2, 3)
plt.plot(x, np.tan(x), 'g-')
plt.title('tan(x)')
plt.ylim(-5, 5)
plt.grid(True)
```

```
# 子图4: 指数衰减
```

```
plt.subplot(2, 2, 4)
plt.plot(x, np.exp(-x/5), 'm-', linewidth=2)
plt.title('exp(-x/5)')
plt.grid(True)
```

```
plt.tight_layout() # 自动调整间距
plt.show()
```



```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(0, 10, 100)
```

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
```

```
# 子图1: 正弦
```

```
axes[0, 0].plot(x, np.sin(x), 'b-')
axes[0, 0].set_title('sin(x)')
axes[0, 0].grid(True)
```

```
# 子图2: 余弦
```

```
axes[0, 1].plot(x, np.cos(x), 'r--')
axes[0, 1].set_title('cos(x)')
axes[0, 1].grid(True)
```

```
# 子图3: 正切
```

```
axes[1, 0].plot(x, np.tan(x), 'g-')
axes[1, 0].set_title('tan(x)')
axes[1, 0].set_ylim(-5, 5)
axes[1, 0].grid(True)
```

```
# 子图4: 指数衰减
```

```
axes[1, 1].plot(x, np.exp(-x/5), 'm-', linewidth=2)
axes[1, 1].set_title('exp(-x/5)')
axes[1, 1].grid(True)
```

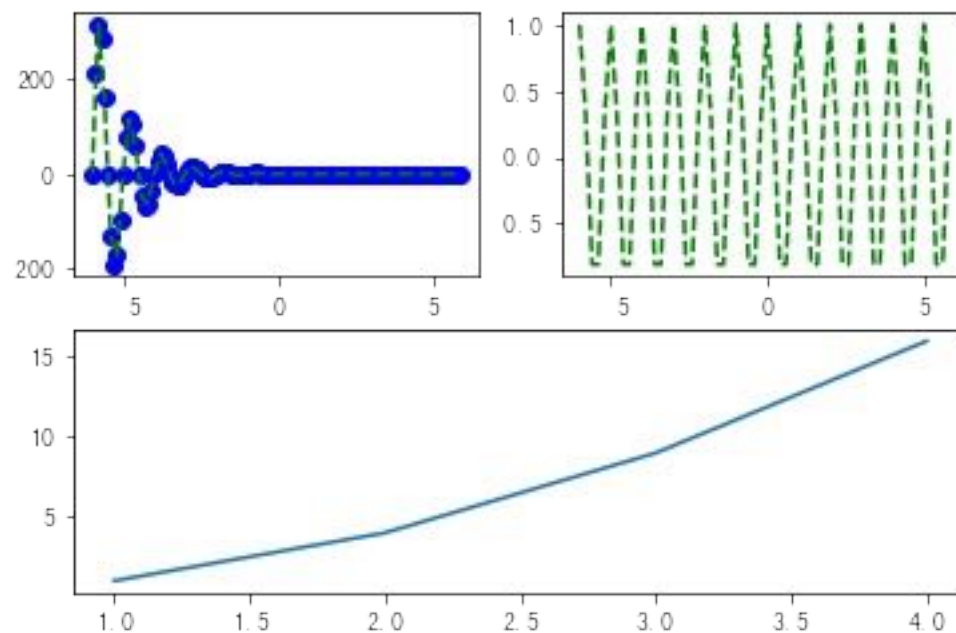
```
plt.tight_layout()
plt.show()
```

案例8

【例】不均匀子图示例。

```
① import matplotlib.pyplot as plt
import numpy as np
② def f(t):
③     return np.exp(-t) * np.sin(2 * np.pi * t)

if __name__ == '__main__':
④     t1 = np.arange(-6, 6, 0.1)
        t2 = np.arange(-6, 6, 0.2)
        plt.figure()
⑤     plt.subplot(221)
        plt.plot(t1, f(t1), 'bo', t2, f(t2), 'g--')
⑥     plt.subplot(222)
        plt.plot(t2, np.cos(2 * np.pi * t2), 'g--')
⑦     plt.subplot(212)
        plt.plot([1, 2, 3, 4], [1, 4, 9, 16])
⑧     plt.show()
```





13.4

案例

案例

【例】 在 matplotlib 中有很多好用的库，常用于金融应用的一个库是 finance 库，matplotlib.finance 子包中有许多绘制金融相关图的函数接口。

由于更新，finance 库已经从 matplotlib 中独立出来，新库名为 mpl_finance。

本节案例使用某只股票从 2011 年 7 月 1 日至 2012 年 6 月 29 日的股价波动情况绘制成 K 线图。股票数据包括：Date（日期）、Open（开盘价）、High（高价）、Low（低价）、Close（收盘价）、Volume（成交量）。

案例 0

```
1 import mplfinance as mpf
2 import matplotlib as mpl
3 from cyclor import cyclor
4 import pandas as pd
5 import matplotlib.pyplot as plt
6
7 def import_csv(stock_code):
8     df = pd.read_csv('Data/' + stock_code + '.csv')
9     df.rename(
10         columns={
11             'date': 'Date', 'open': 'Open',
12             'high': 'High', 'low': 'Low',
13             'close': 'Close', 'volume': 'Volume'},
14         inplace=True)
15     df['Date'] = pd.to_datetime(df['Date'])
16     df.set_index(['Date'], inplace=True)
17     return df
18
19 symbol = 'stockData'
20 period = 100
21 df = import_csv(symbol)
22
23 kwargs = dict(type='candle', mav=(7, 30, 60), volume=True, title='\nA_stock %s candle_line' % (symbol),
24               ylabel='OHLC Candles', ylabel_lower='Shares\nTraded Volume',
25               figratio=(15, 10), figscale=5)
26
27 mc = mpf.make_marketcolors(up='red', down='green', edge='i', wick='i', volume='in', inherit=True)
28 s = mpf.make_mpf_style(gridaxis='both', gridstyle='-.', y_on_right=False, marketcolors=mc)
29
30 mpl.rcParams['axes.prop_cycle'] = cyclor(color=['dodgerblue', 'deeppink', 'navy', 'teal',
31                                               'maroon', 'darkorange', 'indigo'])
32 mpl.rcParams['lines.linewidth'] = .5
33 mpf.plot(df, **kwargs, style=s, show_nontrading=False,
34         savefig='A_stock-%s %s_candle_line' % (symbol, period) + '.jpg')
```

Happy coding! 🐍



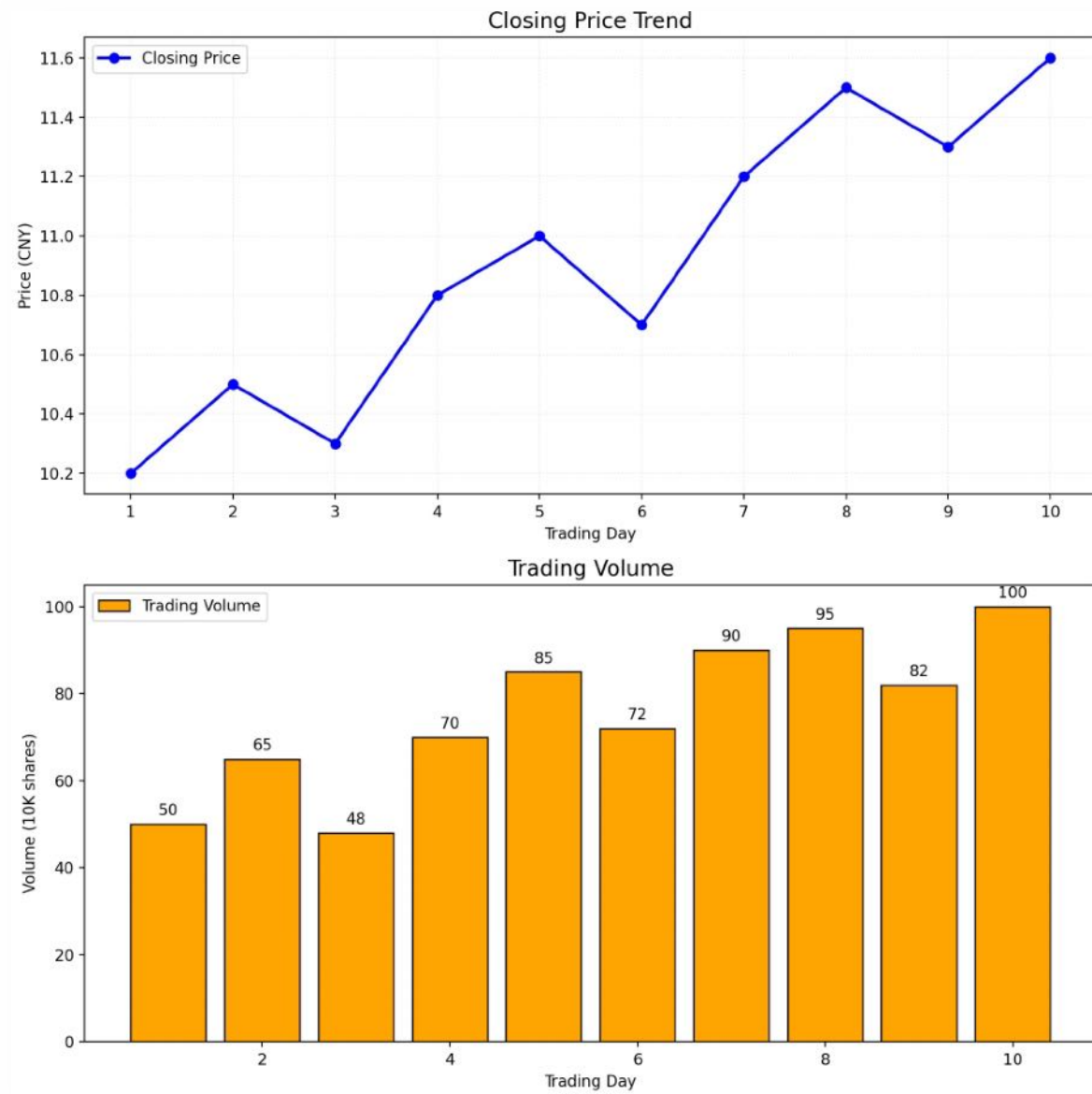
综合练习1：股票数据分析与可视化

某公司过去 10 个交易日的股价数据如表。请用 Matplotlib 完成以下可视化任务

1. 绘制一个 2行1列 的子图，上下排列
2. 上图：绘制收盘价折线图，要求：
 - 蓝色实线，线宽 2
 - 在每个数据点添加红色圆点标记
 - 添加标题“收盘价走势”、X轴标签“交易日”、Y轴标签“价格(元)”
 - 设置 X 轴刻度为 1 到 10
 - 添加网格（虚线，透明度 0.3）
3. 下图：绘制成交量柱状图，要求：
 - 柱子颜色为橙色 (orange)，边框为黑色
 - 添加标题“成交量”、X轴标签“交易日”、Y轴标签“成交量(万手)”
 - 在每个柱子上方标注成交量数值
4. 使用 `plt.tight_layout()` 自动调整布局
5. 将图形保存为 `stock.png`

交易日	1	2	3	4	5	6	7	8	9	10
收盘价(元)	10.2	10.5	10.3	10.8	11.0	10.7	11.2	11.5	11.3	11.6
成交量(万手)	50	65	48	70	85	72	90	95	82	100

综合练习1: 股票数据分析与可视化



综合练习2：随机数据分布与函数曲线对比

某实验测量了 200 个样本的误差值，数据服从正态分布 $N(0, 1)$ 。同时需要对比理论正态分布曲线。请用 Matplotlib 完成可视化任务：

1. 生成 200 个服从 $N(0, 1)$ 的随机数
2. 创建一个 1行2列 的子图，左右排列
3. 左图：绘制散点图，要求：
 - X 坐标为样本序号 (1-200)，Y 坐标为误差值
 - 点的大小为 20，透明度 0.5
 - 根据误差值大小设置颜色（正值红色，负值蓝色）
 - 添加参考线 $y=0$ （黑色虚线，线宽1）
 - 标题“误差分布散点图”，X轴“样本序号”，Y轴“误差值”
4. 右图：绘制直方图，要求：
 - 使用 `hist()` 绘制 20 个 bins
 - 设置颜色为浅蓝色，边框为黑色
 - 叠加理论正态分布曲线（均值=0，标准差=1）
 - 添加图例“实测数据”和“理论分布”
 - 标题“误差直方图与理论曲线”
5. 调整子图间距，使布局美观
6. 添加总标题（整个图形的标题）

综合练习2：随机数据分布与函数曲线对比

Experimental Error Analysis Report

